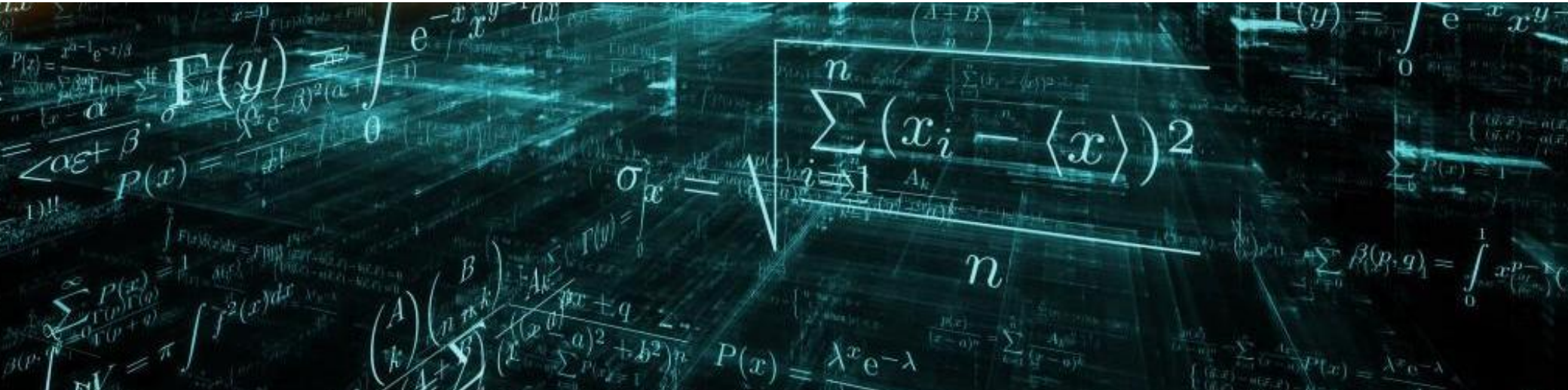
Multimedia Retrieval

Chapter 6: Vector Search

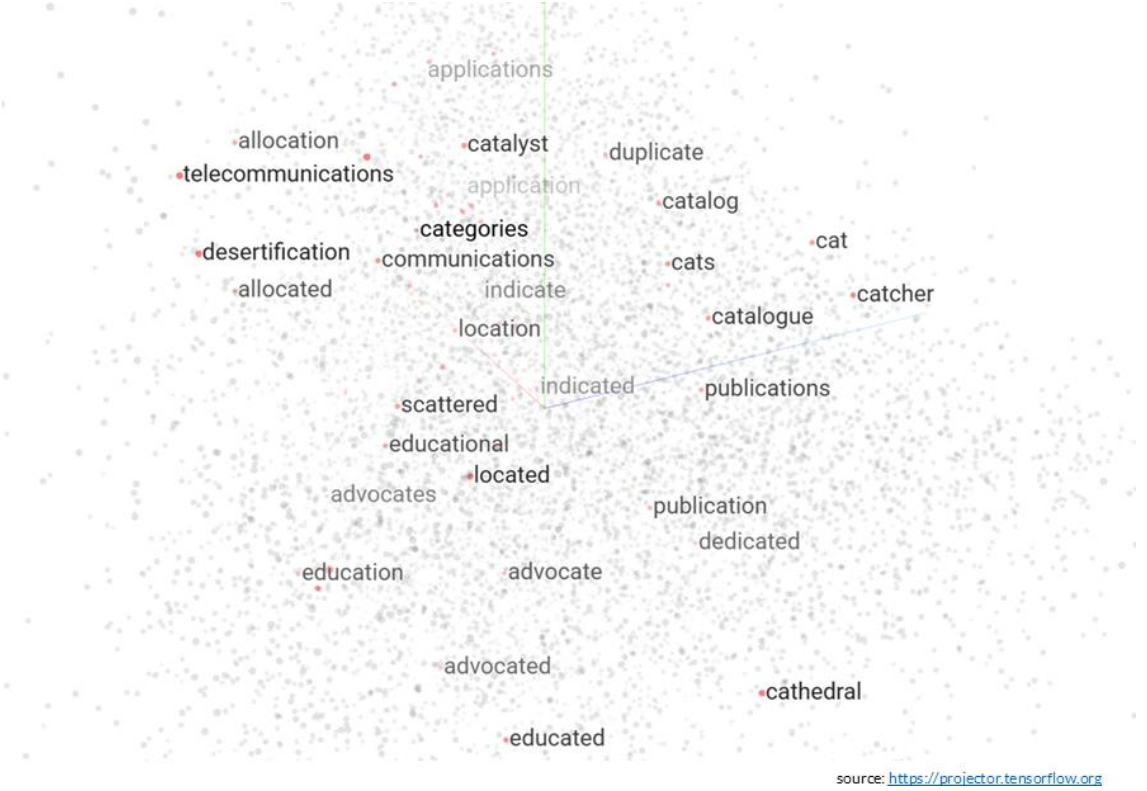
Dr. Roger Weber, roger.weber@gmail.com

6.1 Distance and Similarity
6.2 Low-Dimensional Search Structures
6.3 The Curse of Dimensionality
6.4 Approximate Nearest-Neighbor Search
6.5 Vector Search in Practice



From embeddings to geometry

- Embeddings (ch5, also images, audio, video): related items land **close together**
  - retrieval becomes geometric: which stored points lie closest to the query?
- Dense vectors break the inverted file
  - TF-IDF "heart disease": 2 non-zero components, 2 posting lists
  - embedding: 768 non-zero components; **no document can be skipped**
  - so far: a linear scan over all vectors
- **Nearest-neighbor problem**: organize vectors so a query visits only a small part of the collection
  - solved in 2D/3D decades ago; hard with hundreds of dimensions

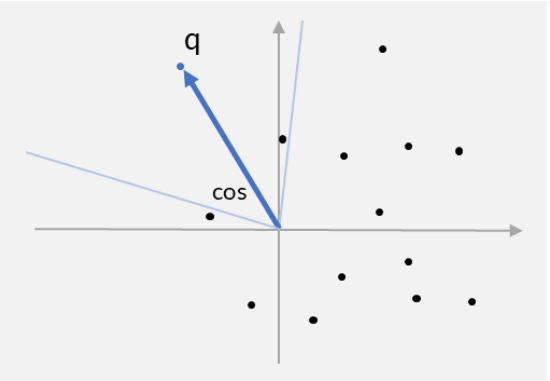


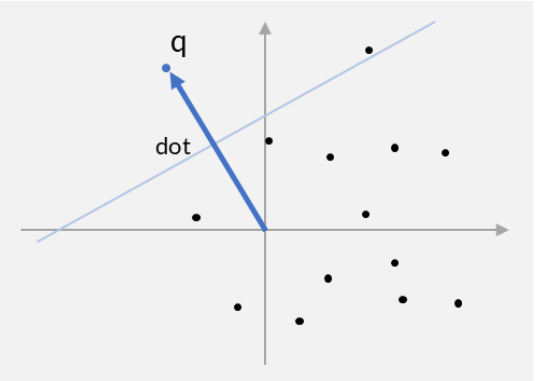
Similarity scores: dot product and cosine

- **Dot product**: sums the products of matching components
  - rewards vectors that are both **aligned and long**
  - best matches lie beyond a hyperplane orthogonal to q
- **Cosine**: divides out both lengths
  - rewards **alignment alone**; best matches lie inside a cone around q
  - documents in the opposite half-space score negatively under both
- Both: larger is better

$$sim_{dot}(\boldsymbol{q}, \boldsymbol{d}_i) = \boldsymbol{q} \cdot \boldsymbol{d}_i = \sum_{k=1}^{M} q_k \; d_{i,k}$$

$$sim_{cos}(\boldsymbol{q}, \boldsymbol{d}_i) = \frac{\boldsymbol{q} \cdot \boldsymbol{d}_i}{\|\boldsymbol{q}\| \, \|\boldsymbol{d}_i\|}$$

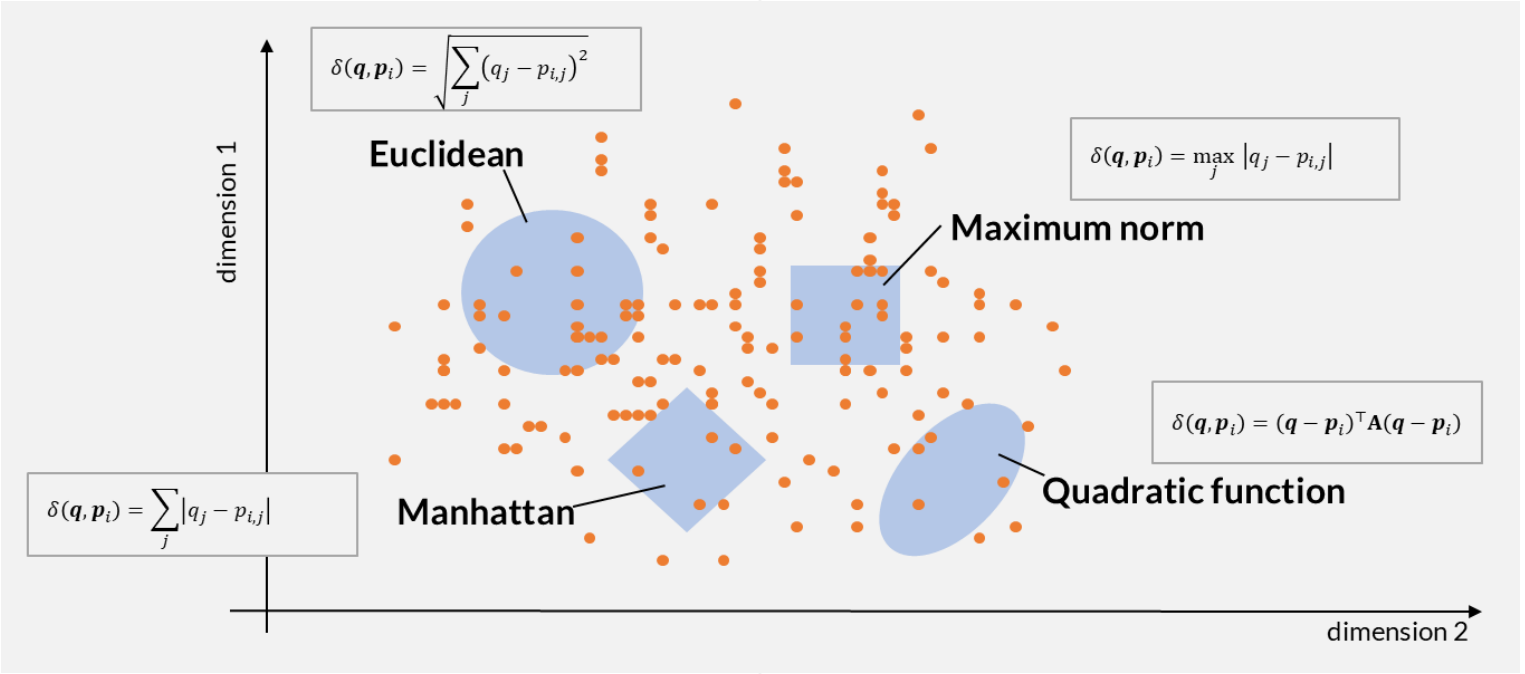




Distances: smaller is better

- **Perceptual search**: media described by extracted features
  - e.g. color histogram: pixels per color bin, 48 dims
  - compared by a **distance**: similar colors, small L1 sum
- **Distance** δ(q, p): smaller = better, 0 if identical; each induces a **neighborhood shape**

- **Euclidean** (L2): straight line, round; default for continuous features
- **Manhattan** (L1): sum of per-dimension differences, diamond; emphasizes many small differences
- **Maximum** (L-infinity): largest single difference, square; worst dimension dominates
- **Quadratic**: weights and correlates dimensions through a matrix, tilted ellipse



The common search problem: best match and nearest neighbor

- **One search problem**, two directions
  - best match: maximize a similarity (top-k)
  - nearest neighbor: minimize a distance (k-NN)
  - neither says **how**; a linear scan computes all

- **Convert** between similarity and distance
  - cosine distance 1 - cos (pgvector <=>)
  - Lucene: (1 + cos) / 2, in [0, 1]
  - any distance: 1 / (1 + δ), same ranking
  - **replacing** the comparison changes the ranking

- Normalize documents at index time: cosine = dot product

$$\boldsymbol{d}^* = \operatorname{argmax}_{\boldsymbol{d}_i \in \mathbb{P}} sim(\boldsymbol{q}, \boldsymbol{d}_i), \qquad sim \in \{sim_{dot}, sim_{cos}\}$$

$$NN(\boldsymbol{q}) = \operatorname{argmin}_{\boldsymbol{p}_i \in \mathbb{P}} \delta(\boldsymbol{q}, \boldsymbol{p}_i)$$

$$\delta_{\cos}(\boldsymbol{q}, \boldsymbol{d}_i) = 1 - sim_{\cos}(\boldsymbol{q}, \boldsymbol{d}_i)$$

$$sim(\boldsymbol{q}, \boldsymbol{p}_i) = \frac{1}{1 + \delta(\boldsymbol{q}, \boldsymbol{p}_i)}$$

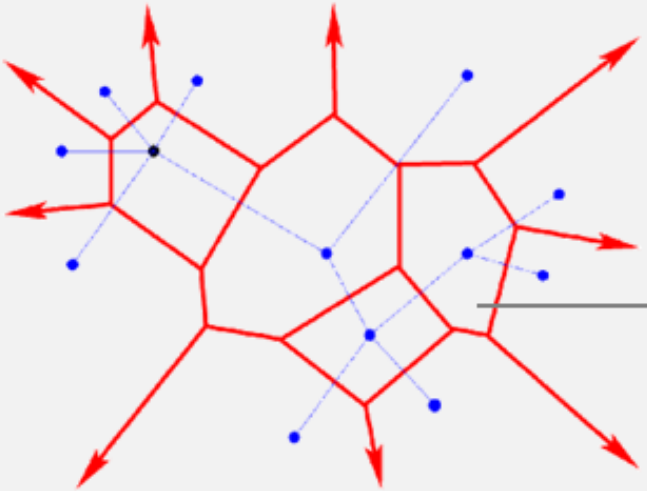
Precomputing all answers: Voronoi diagrams

- **Post-office problem** (Knuth): the nearest post office to home, NN search in 2D

- **Voronoi diagram**: one cell per point, all locations closer to it than to any other
  - query: locate the cell containing q
  - precomputes the answers to **all** queries, like the inverted file per term

- **Not practical beyond a few dimensions**
  - worst-case size ~ n^(d/2): quadratic already in 3D
  - bounding-rectangle approximations overlap
  - insertions reshape all neighboring cells

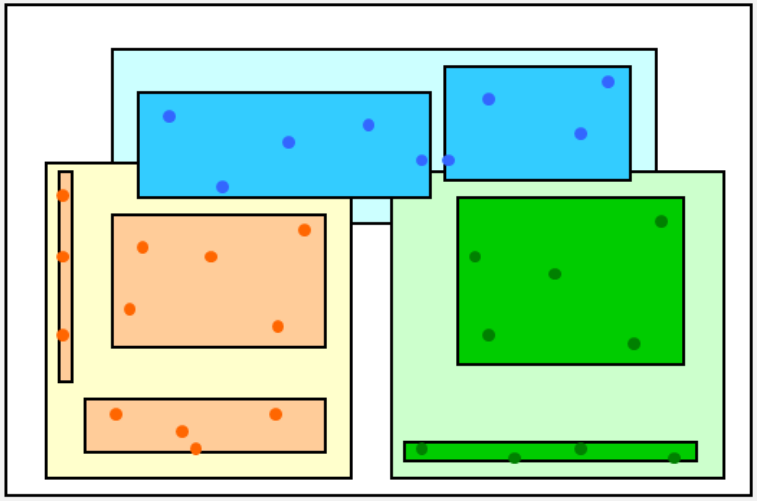
- Still the clearest **mental model** of exact NN search

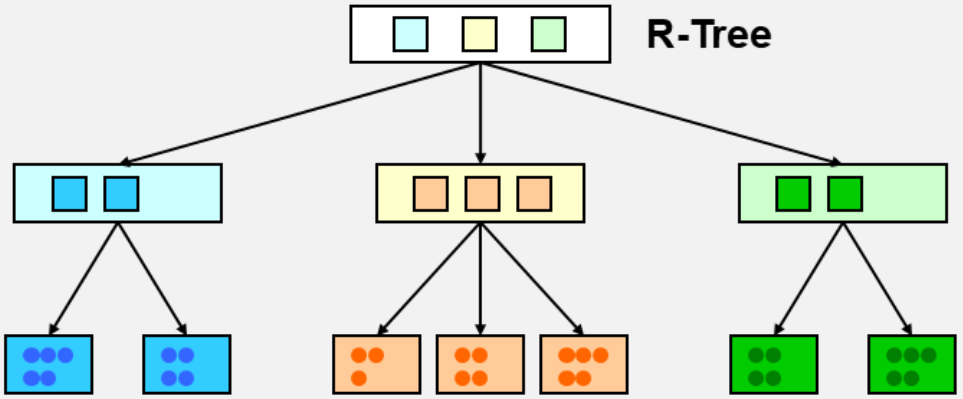


Voronoi Cell: all points in the cell are closest to blue center point

R-trees: the B-tree in many dimensions

- **R-tree**: nested **minimum bounding rectangles** (MBRs) up to one root
  - balanced like a B-tree; nodes split on overflow
  - points, rectangles, polygons; a mainstay of spatial databases
- **NN search is not range search**
  - radius unknown: find a candidate, then **prove** no point is closer
  - the true NN may sit in another subtree
  - several root-to-leaf paths, even without overlap





Optimal NN search: inspect every node that intersects the NN-sphere

- Priority queue of nodes and points by distance to q (Hjaltason, Samet)
  - node: **lower bound** for any point inside
- **Key point**: the NN-sphere around q, through the true NN
  - every node intersecting it **must** be inspected
  - the algorithm inspects exactly these: **optimal**
- Example: q = (4.6, 4.6), 4 leaves
  - L3 looked best (1.00); NN (3.8, 3.4) at 1.44 is in the other subtree
  - L2, L4 skipped (bounds > 1.44)

```
insert the root into the queue with its lower bound
while the queue is not empty:
    remove the entry e with the smallest distance
    if e is a data point:  return e as the nearest neighbor
    if e is a leaf:        insert every point of e
                           with its distance to q
    if e is an inner node: insert every child of e
                           with its lower bound
```

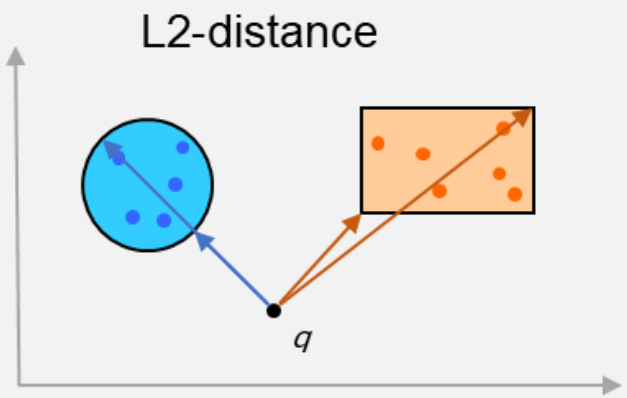
Step	Removed	Inserted
1	root	B (0.60), A (0.80)
2	B (0.60)	L3 (1.00), L4 (1.97)
3	A (0.80)	L1 (1.44), L2 (1.89)
4	L3 (1.00)	(6.5, 5.4) 2.06, (5.2, 7) 2.47, (8, 9) 5.56
5	L1 (1.44)	(3.8, 3.4) 1.44, (3, 1.5) 3.49, (1, 1) 5.09
6	(3.8, 3.4) 1.44	none: NN found

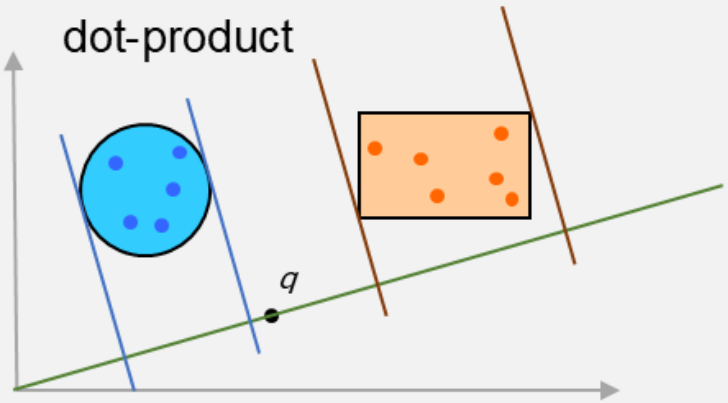
Beyond Euclidean: dot product and cosine

- The algorithm never looks inside the distance: it needs an **optimistic bound** per region
  - distance: lower bound, ascending queue
  - similarity: upper bound, descending queue

- **Dot product** = sum of products
  - q_j p_j is largest at u_j if q_j ≥ 0, else at l_j
  - ball (center c, radius r): q·c + r‖q‖ (Ram and Gray)

$$uBnd(\boldsymbol{q}, MBR) = \sum_j \begin{cases} q_j\, u_j & q_j \geq 0 \\ q_j\, l_j & q_j < 0 \end{cases}$$





Distances lose their contrast

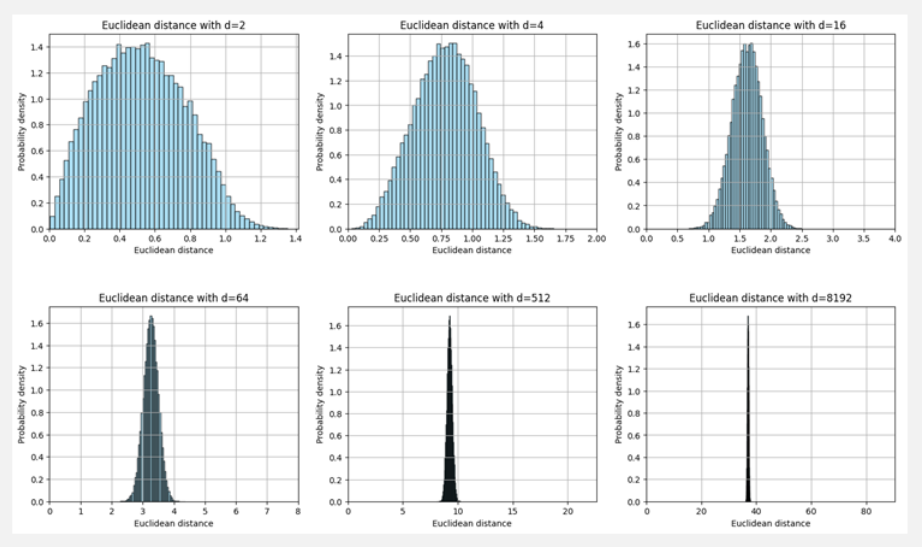
- Distances to a query grow **at the same pace** for all points, so they become ever more similar
  - **contrast**: how much farther the farthest point is than the nearest

- Experiment: 10,000 uniform random points in the unit cube, 50 queries
  - 2D: farthest hundreds of times farther than nearest
  - d = 1000: only 17% (L1) or 14% (L2) farther
  - mean distance grows like √d, its spread stays at about 0.23
  - L1 keeps a little more contrast than L2; both collapse

- d = 8192: distances pile up around 36.9 with std. dev. 0.24; 99% of pairs in a band barely one unit wide

$$\frac{\delta_{\max} - \delta_{\min}}{\delta_{\min}} \to 0 \qquad (d \to \infty)$$

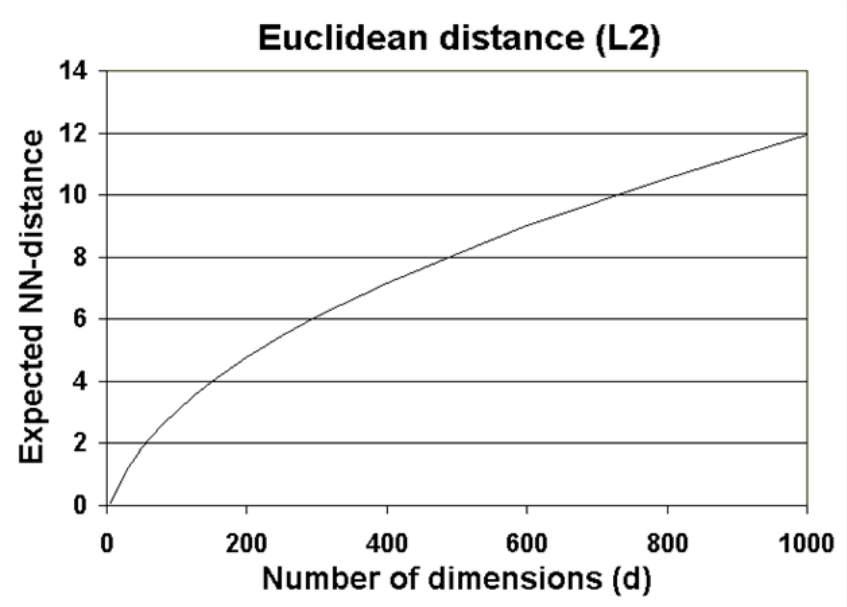
d	Contrast L1	Contrast L2	Mean L2	Std. dev. L2
2	401	355	0.53	0.23
10	5.2	4.2	1.28	0.23
100	0.69	0.54	4.10	0.23
1000	0.17	0.14	12.90	0.22

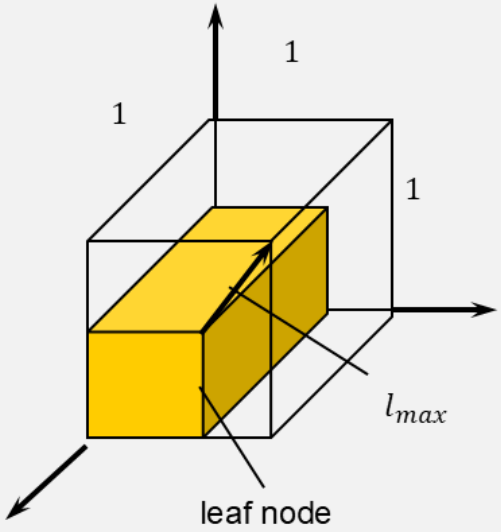


Consequence 1: trees can no longer prune

- Model: uniform unit cube, L2, 10^6 points
- **Expected NN distance** grows: 0.24 at d = 10, 1.37 at 40, 2.97 at 100
- **Leaf size** is bounded: ~14 halving splits, so a leaf spans d' ≈ 14 dimensions; no query is farther than l_max = 0.5√d' ≈ 1.87

- NN distance > l_max: **every leaf** hits the NN-sphere (d = 40 to 60)
  - even the optimal algorithm reads everything
- In practice a scan wins beyond ~**10 dimensions** (Weber, Schek, Blott)
  - sequential reads beat leaf-by-leaf random reads



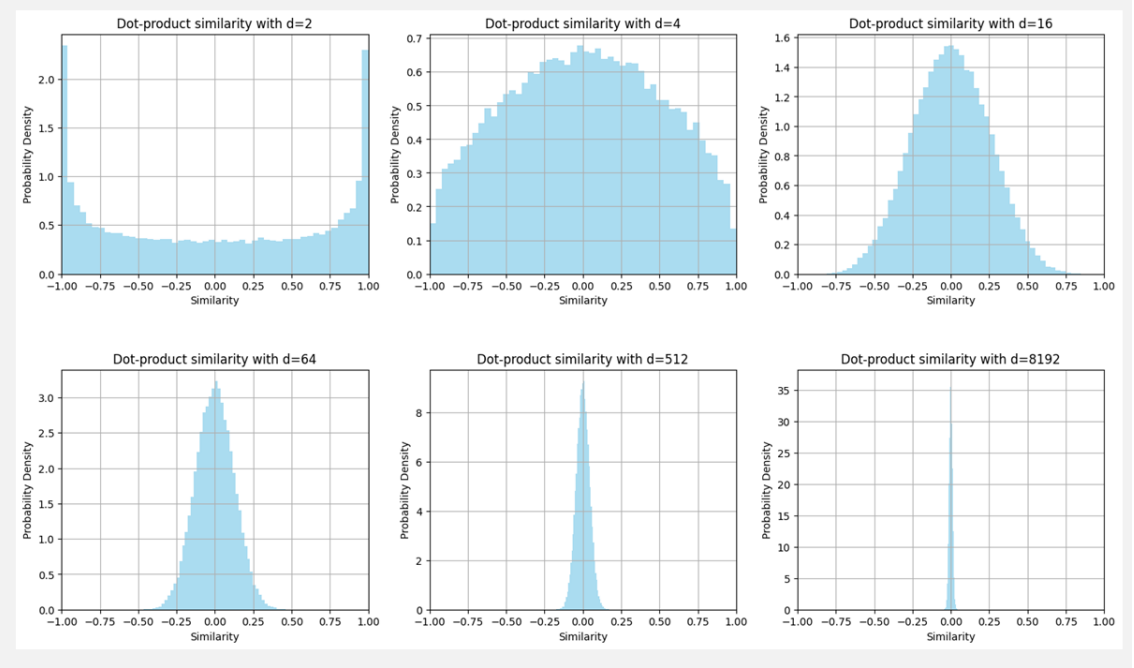


Dot products concentrate around zero

- Random components in [-1, 1]: each product x_k y_k has mean 0, variance 1/9
  - dot product: mean 0, std. dev. √d / 3
  - lengths grow faster: each ≈ √(d/3), product ≈ d/3; dividing gives a shrinking cosine spread

$$\sigma_{\cos} \approx \frac{1}{\sqrt{d}}$$

- **Random vectors become almost orthogonal**
  - d = 768: σ ≈ 0.036; best of 10^6 random vectors reaches cosine ≈ 0.17
  - d = 8192: σ ≈ 0.011; good and bad matches differ in the third decimal



- Unnormalized dot product: relative spread also 1/√d
  - lengths concentrate too (4.5% at d = 100, 1.6% at d = 768)
  - correlation dot vs cosine: 0.88 at d = 2, 0.9997 at d = 768

Good matches instead of the best match

- **Consequence 2: what does "nearest" still mean?**
  - all points almost equally far: the NN is only marginally nearer
  - a point 14% farther is not meaningfully less similar
- **Real data is kinder**: it forms clusters
  - sunsets near sunsets, cardiology near cardiology
  - cluster members stand out from the uniform background as outliers with clearly smaller distances (or higher cosines)
  - this is why vector search works at all
- **Within a cluster** the loss of contrast remains
  - top few candidates almost equally close; the exact NN is one good match among several
  - a full scan to decide which is nearest buys a distinction with little meaning
- **Retriever-reranker pipelines** relax further
  - the retriever only needs the good hits among its candidates; the reranker restores precision
- So: give up exactness **on purpose** and search approximately for good matches

▶ **Demo**: ch06-01-distances-and-dimensionality.ipynb

Three levers to accelerate search

- Why approximation is acceptable
  - first stage of a pipeline: good matches among the candidates suffice; the second stage orders them
  - even without a reranker: the third-best instead of the best hardly changes what the user sees

- Cost of a full scan = **number of vectors x size of each vector**
  - 10^9 embeddings x 768 dimensions x 4 bytes ≈ **3 TB read per query**

- Three levers, each trading a little accuracy for speed
  - combined: two to three **orders of magnitude** less work per query

Lever	Reduces	Methods
1. Fewer dimensions	size of each vector	PCA, Matryoshka truncation
2. Fewer bits per vector	size of each vector	scalar quantization, PQ, LSH
3. Fewer vectors read	number of vectors	IVF, HNSW

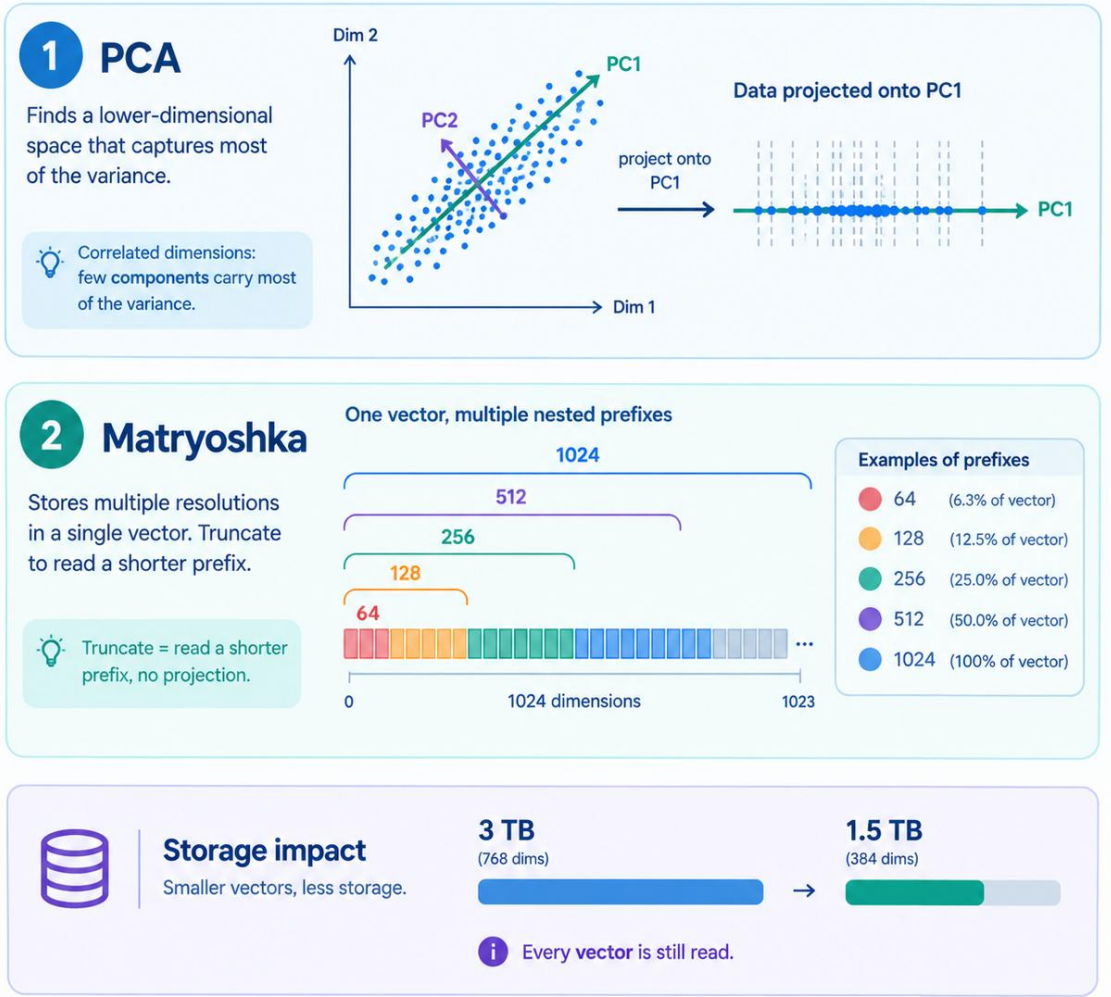
How much does approximation speed up search, and what does it cost in quality? How do we measure the quality of an approximation? Answered on the recall and TREC-COVID slides.

Lever 1: fewer dimensions

- **PCA**: project onto the directions of largest variance
  - works when dimensions are correlated, e.g. neighboring histogram bins
  - distances in the reduced space approximate the originals

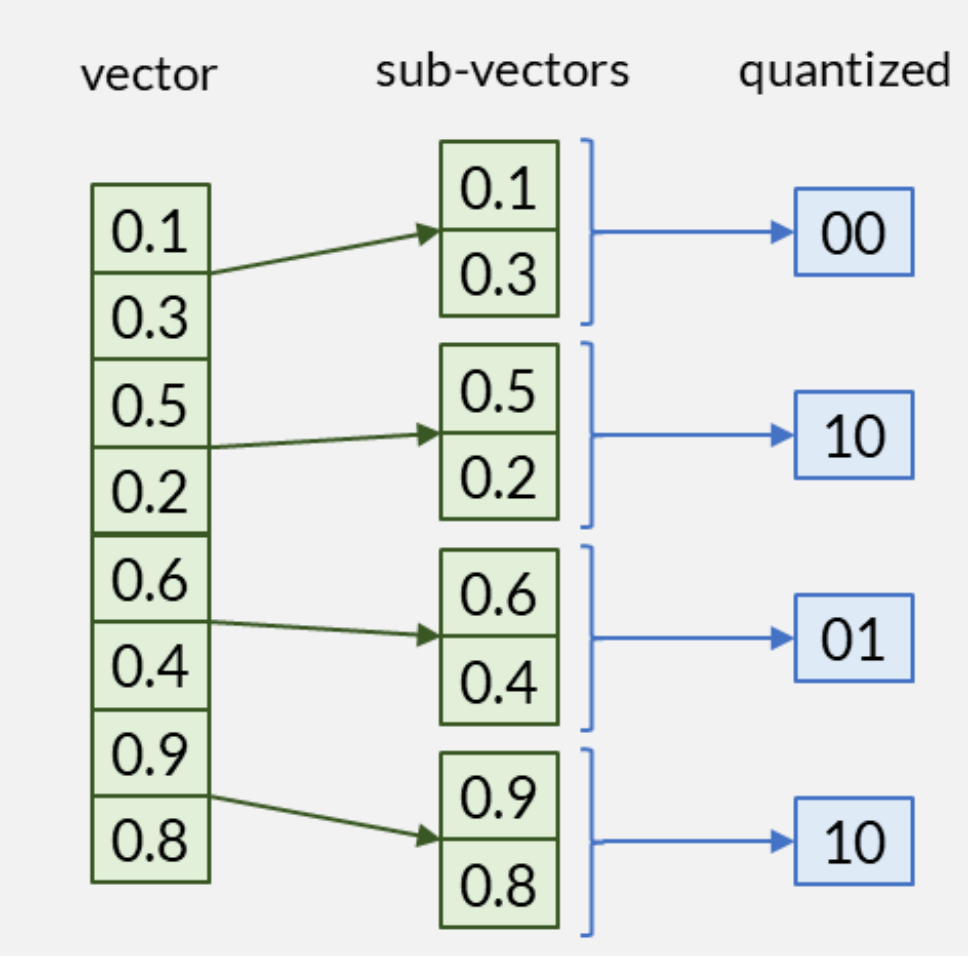
- **Matryoshka** embeddings: leading dimensions carry most information
  - a prefix is itself an embedding: truncate, no projection

- **Modest lever**: removes ~20 to 60% of the data
  - 768 to 384 dimensions: 3 TB to 1.5 TB
  - every vector is still read: a cheaper full scan



Lever 2: product quantization

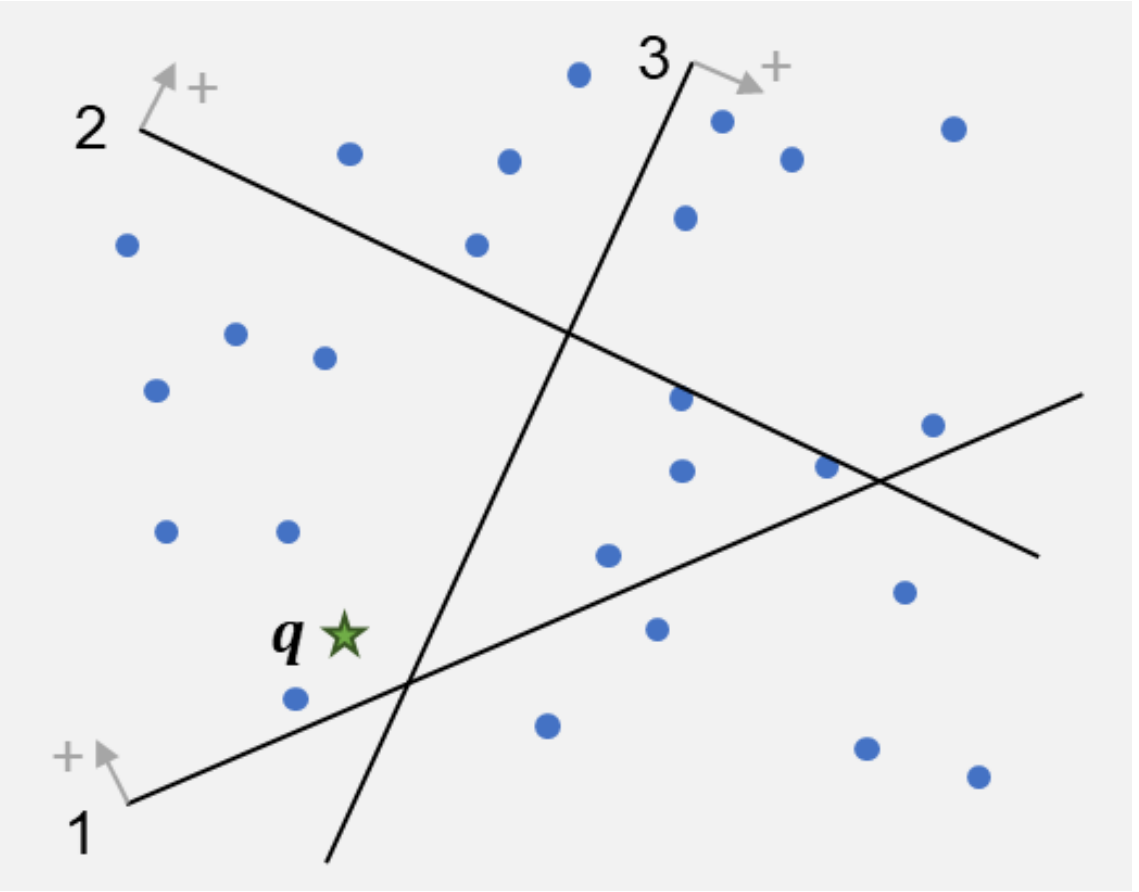
- **PQ** (Jégou et al. 2011): quantize **groups** of dimensions
  - m sub-vectors, each sub-space with its own **codebook** of 2^b centers (k-means)
  - store the nearest center's number per sub-vector
  - figure: 8 floats (256 bits) to 8 bits, 1:32
- **Lookup tables**: per query m tables of 2^b distances; a distance = m look-ups
- **10^9 x 768d**, m = 96, b = 8
  - 96 bytes instead of 3,072: 96 GB instead of 3 TB
  - tables cost about 256 full comparisons, once per query
  - per vector: 96 look-ups, not 768 multiply-adds



$$\tilde{\delta}(\boldsymbol{q},\boldsymbol{p})^2 = \sum_{j=1}^{m} \left\| \boldsymbol{q}_j - \boldsymbol{c}_j(\boldsymbol{p}) \right\|^2$$

Lever 2: locality-sensitive hashing

- **LSH**: h hyperplanes, one bit each: 1 on the positive side
  - figure: 3 lines, 7 regions; q gets code 100
  - neighboring regions differ in one bit
- Buckets fail for large h (2^{64} codes, mostly empty)
  - rank all codes by **Hamming distance**: XOR + popcount
  - h = 256: 3,072 to 32 bytes (96x); ties broken by exact reranking
- Hamming distance tracks the **angle** (Charikar 2002)
  - serves cosine; dot product via the extension trick
  - finds the cluster reliably, **not the order within it**



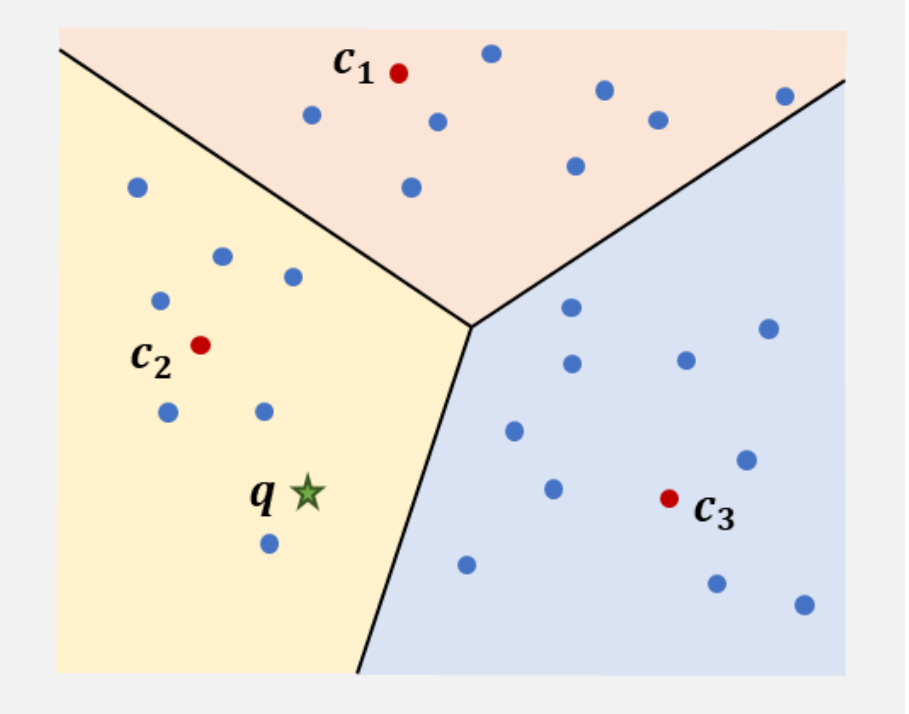
$$P(\textit{bit differs}) = \frac{\theta}{\pi} \qquad E[\textit{Hamming}] = h \cdot \frac{\theta}{\pi}$$

Lever 3: the inverted file (IVF)

- Visit only the **most promising** regions; accept missing a neighbor just across a boundary

- **IVF** borrows the inverted index
  - k-means: n_list centers, each vector in its nearest center's list
  - query: compare with all centers, scan the n_probe nearest lists
  - share read = n_probe / n_list; 4,096 lists, 16 probes: 1/256

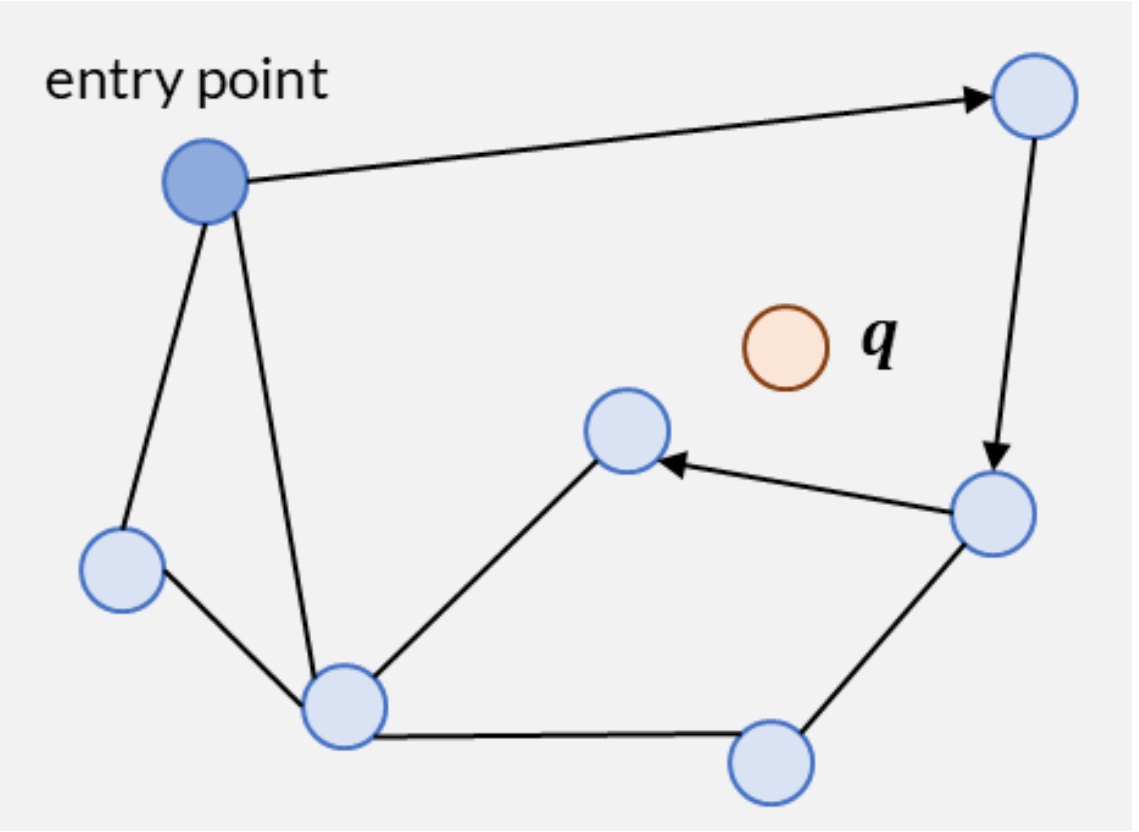
- **n_probe is the tuning knob**: cost linear, recall gains shrink
  - 100 to 200 probes: half the QPS, +4 recall points
  - 200 probes still 18x faster than the full scan (16.5 QPS)



Probed lists	Share read	Recall@10	QPS
5	0.1%	0.59	4,929
10	0.2%	0.68	3,443
50	1.2%	0.84	1,003
100	2.4%	0.89	548
200	4.9%	0.93	298

Lever 3: navigable small-world graphs (HNSW)

- **NSW graph**: links to nearest neighbors ("friends") plus shortcuts
  - **navigable**: a step toward q makes progress
  - **small world**: short paths between any two nodes
- **Greedy search**: go to the neighbor closest to q until none is closer
  - local minimum: approximate
  - keep the best **ef_search** candidates, not one
- **HNSW**: layers, random subsets toward the top
  - long jumps on top, refine at the base; **M** links per node
  - price: ~256 B of links per vector, all **in RAM**



Combining the levers: FAISS composite indexes

- No single lever wins: FAISS stacks them; the index string names its stages in order

- 10^9 x 768d: **PCA384,IVF4096,PQ48,RFlat**
  - codes: 48 GB, fit in memory
  - 16 probes: 3.9M codes ≈ 190 MB per query
  - rerank k' = 100: 300 KB exact vectors
  - ~16,000x less data than 3 TB; real speed-up smaller

- **Errors compound** as well as gains
  - PCA loses information, PQ distorts, IVF misses lists
  - tune all parameters **together**, ideally against relevance judgments

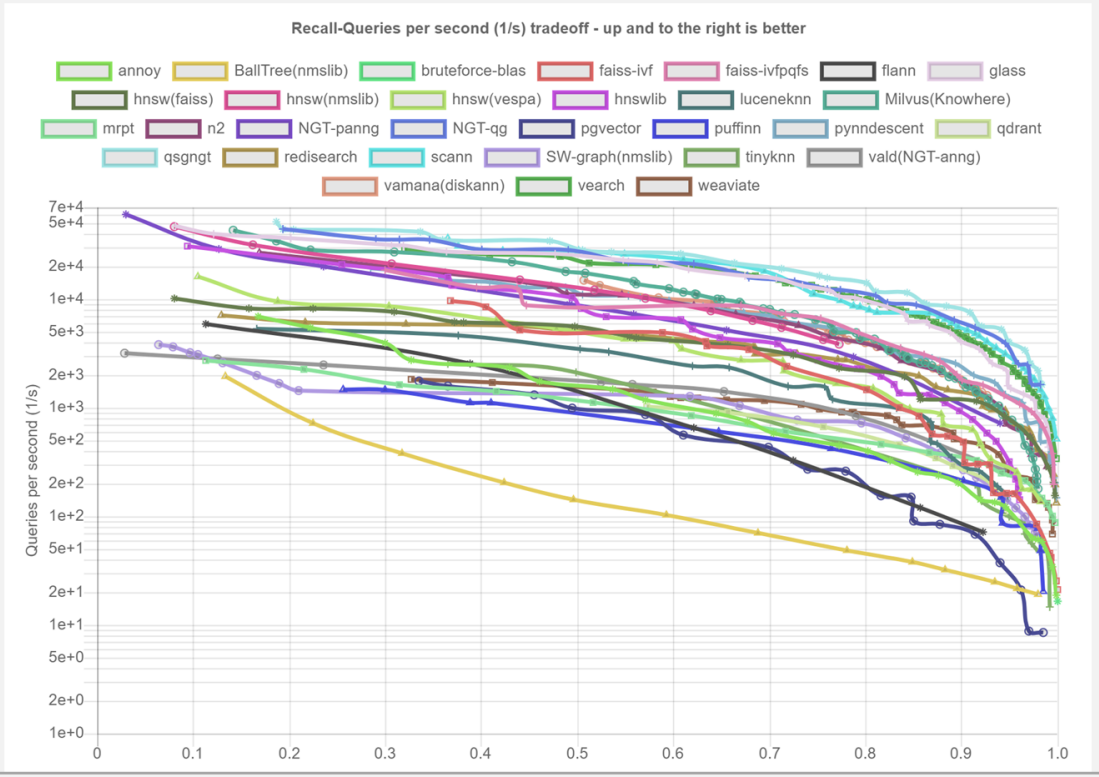
Stage	Lever	Options	In the example
Vector transformer	1. fewer dimensions	PCA, L2-normalization	PCA384
Coarse quantizer	3. fewer vectors read	IVF, HNSW	IVF4096
Fine quantizer	2. fewer bits per vector	SQ, PQ, LSH	PQ48
Refiner	exact rerank, filters	exact distances on k'	RFlat

Measuring the trade-off: recall versus throughput

- Every method has a knob trading recall for speed
- Measure both: **recall@k** vs exact search, **throughput** (QPS)

$$recall@k = \frac{|approximate\ top{-}k \cap exact\ top{-}k|}{k}$$

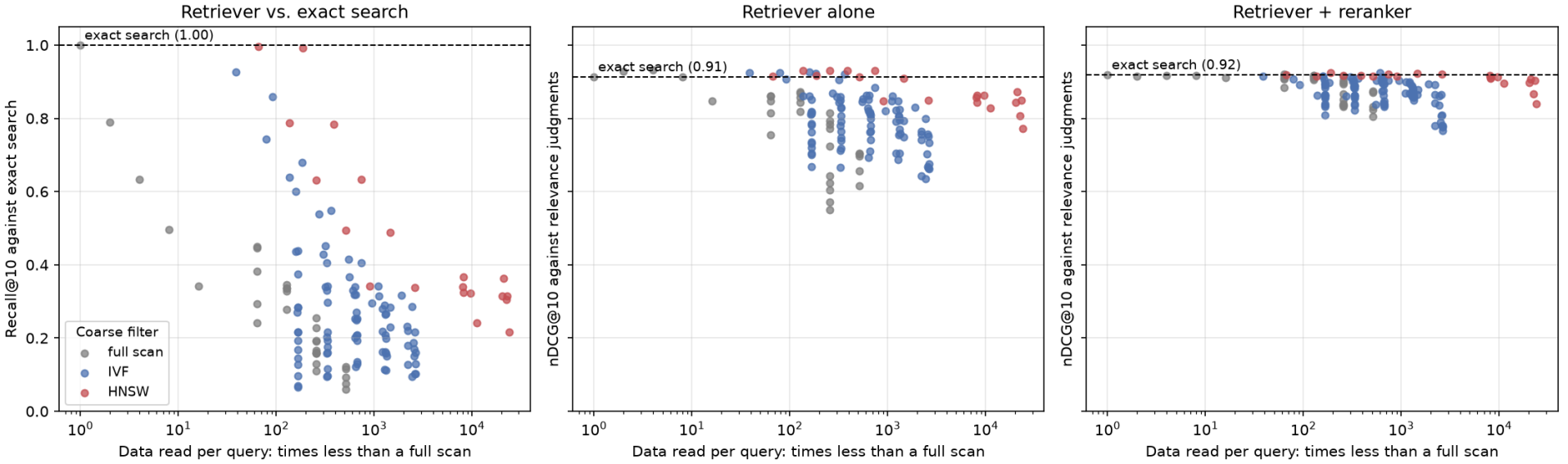
- ann-benchmarks, GloVe-100 (1.2M, cosine, k = 10)
  - fix the recall you need, pick the highest curve
  - full scan 16.5 QPS; best 850x faster at 0.80, 240x at 0.95
  - the last points of recall are the most expensive
- Not shown: memory and build time (HNSW in RAM; PQ small)



▶ **Demo**: ch06-02-approximate-nn.ipynb

Recall understates quality: 140 retrievers, three scores

- **One dot per retriever**, the same 140 retrievers in every panel (TREC-COVID), scored three ways; x-axis: data read, times less than a full scan
  - left, **recall@10 vs exact search**: median 0.24, approximate search looks like a failure
  - middle, **nDCG@10 alone**: median 0.80 (exact 0.91), the returned documents are often just as relevant
  - right, **nDCG@10 after reranking its top 50**: median 0.88 (exact 0.92)

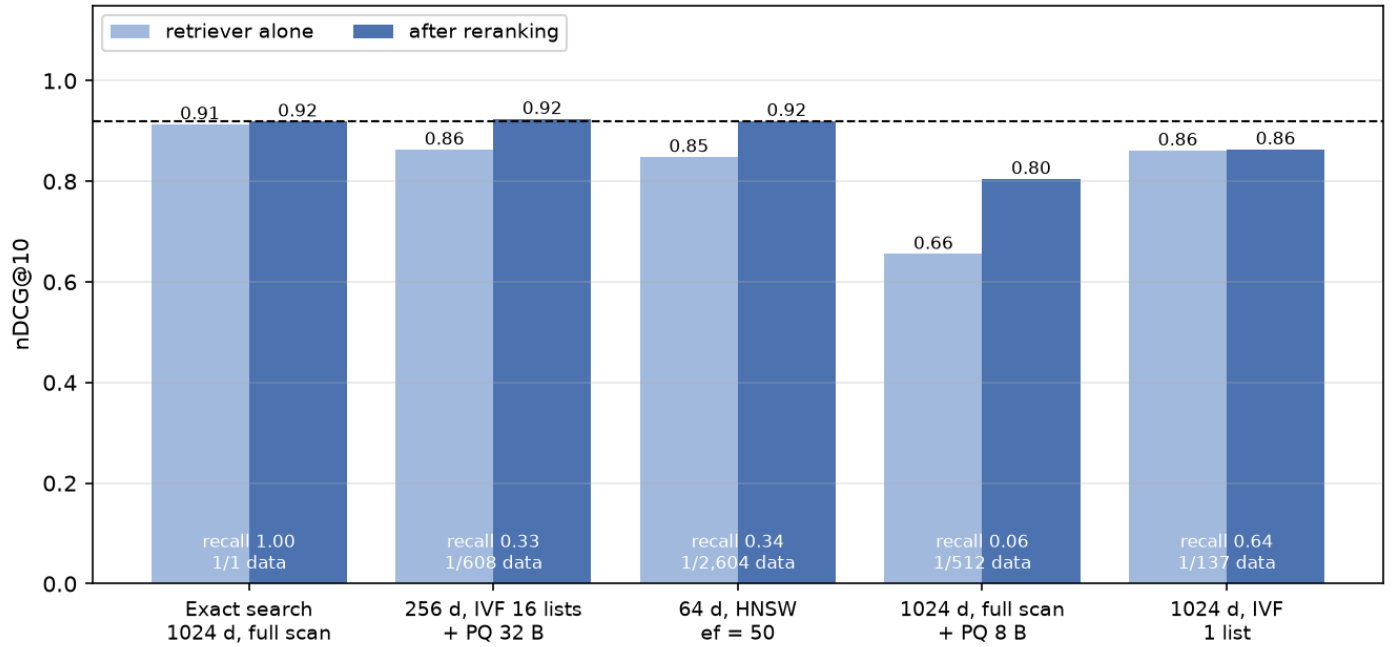


Which errors a reranker repairs: five retrievers

- Five approximate retrievers from the previous slide: nDCG@10 alone vs after reranking
  - **repaired to exact**: IVF 16 lists + PQ 32 B (608x less data, recall 0.33); HNSW on 64d (2,604x less)
  - **partly**: PQ 8 B, 0.66 to 0.80
  - **not**: IVF with 1 list stays at 0.86

- **Fine-filter errors are repairable, coarse-filter errors are not**
  - unread clusters never reach the reranker
  - keep the coarse filter generous

- Default: HNSW if it fits in RAM, else IVF + PQ + rerank



FAISS: a library of vector indexes

- Running query: "a mind-bending heist thriller with Leonardo DiCaprio, after 2005, rated 7.5 or better"

- C++ library (Meta), Python bindings, CPU and GPU
  - all methods of the previous section, combined via **index strings**
  - new method = new string: HNSW32, IVF256,PQ64,RFlat

- Stores **only vectors and integer ids**; metadata stays in the application

```python
import faiss
import numpy as np

# 1. Movie embeddings (one row per movie) and their movie ids
X = np.load("overview_embeddings.npy").astype("float32") # (n,1024)
ids = np.load("movie_ids.npy").astype("int64")
faiss.normalize_L2(X)   # unit length: inner product = cosine

# 2. Build an IVF index with PQ codes (lever 3 + lever 2)
index = faiss.index_factory(1024, "IVF256,PQ64",
                            faiss.METRIC_INNER_PRODUCT)
index.train(X)          # k-means for the centers and the codebooks
index.add_with_ids(X, ids)

# 3. Search: embed the query and probe 16 of the 256 lists
q = embed("a mind-bending heist thriller").reshape(1, -1)
faiss.normalize_L2(q)
index.nprobe = 16
scores, movie_ids = index.search(q, 10)
```

▶ **Demo**: ch06-03-ann-benchmark.ipynb

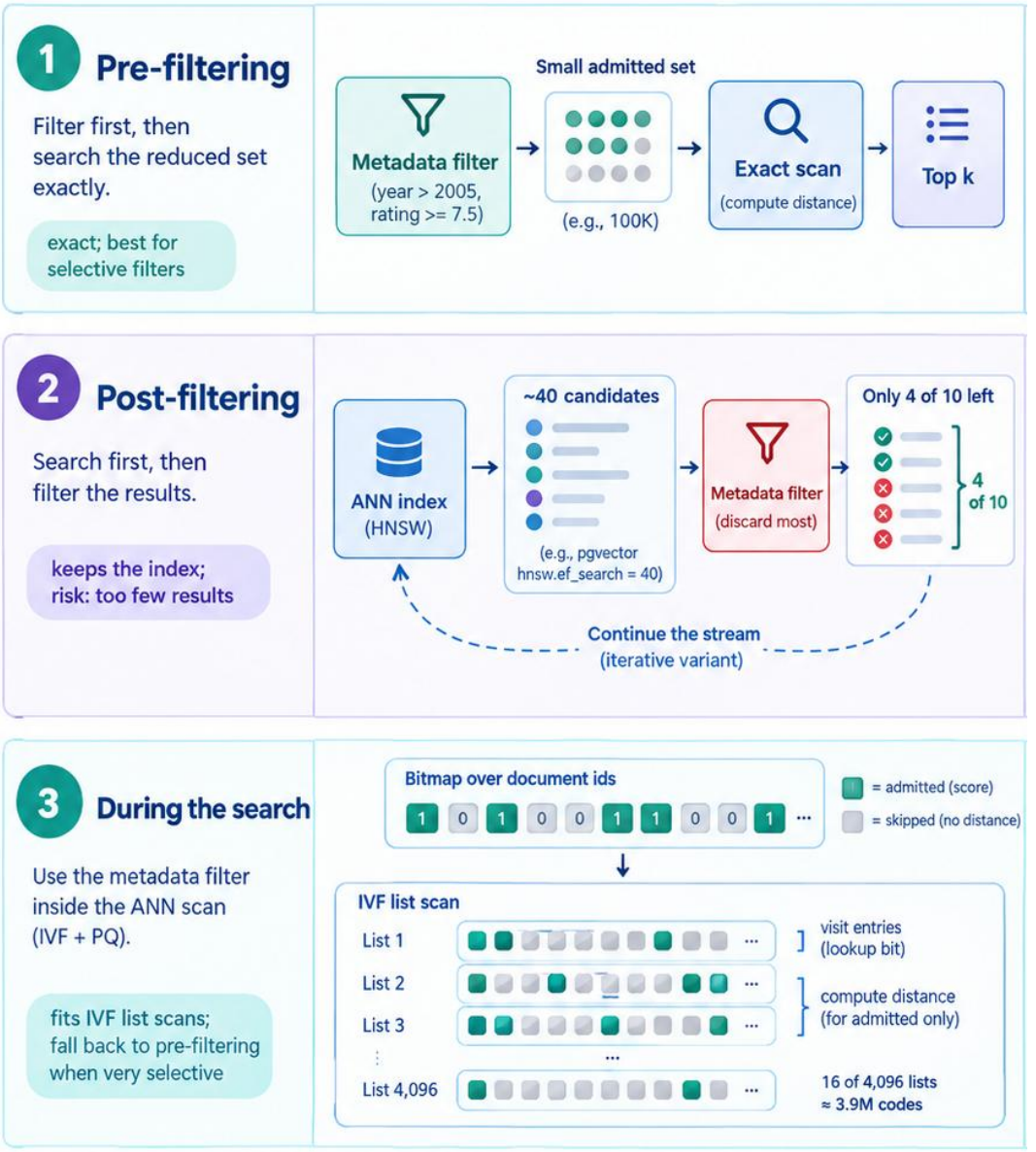
Four systems, three levers

- Same methods, exposed in different ways; every system can still search exactly
- Realistic queries need **three ingredients**: vector search for meaning, keyword search for names and terms, filters for hard constraints

Lever	FAISS	Lucene	OpenSearch	pgvector
Fewer dimensions	PCA stage in the index string	truncation by the application	truncation by the application	index on a subvector
Fewer bits per vector	scalar, PQ, LSH	7, 4, or 1 bit per component	scalar, binary, PQ (FAISS engine)	halfvec, binary_quantize
Fewer vectors read	IVF, HNSW	HNSW per segment	HNSW, IVF (FAISS engine); shards	HNSW, IVFFlat
Exact search	Flat index	yes	yes	yes, without an index

Three ways to filter

- The ANN index knows **nothing about metadata**
- **Pre-filtering**: filter, then exact scan of the survivors
  - exact; cheap if the filter is **selective**
- **Post-filtering**: ANN first, discard failing results
  - risk: **too few results**; robust: consume a candidate stream until k pass
- **During the search**: bitmap of admitted ids
  - fits IVF list scans: one bit lookup per entry



The limits of filtering during the search

- **Very selective filters** break the IVF scan
  - 0.001% admitted: the probed lists hold only ~39 admitted vectors
  - their best ten may be poor; the good admitted matches sit in unprobed lists
  - the 10,000 admitted vectors fit an exact scan of 30 MB: cheaper **and** exact
- So systems **choose per query**: few admitted documents, skip the index and pre-filter

- **Graph indexes** can filter during the search too
  - but must still step through non-matching nodes on the way to matching ones
  - with a selective filter, most graph steps are wasted
- **Rule of thumb**
  - pre-filtering: selective filters
  - post-filtering on a candidate stream: graph indexes
  - during the search: list scans of an inverted file

▶ **Demo**: ch06-04-vector-search-frameworks.ipynb

Hybrid search with Lucene

- **Filter during the search**: 4th argument of KnnFloatVectorQuery
  - few admitted documents: exact search instead
- Keywords: BM25 clauses of a Boolean query
- Two pitfalls
  - filter outside the k-NN query = **post-filtering**
  - Boolean query **adds** scores: BM25 dominates

```java
// 1. Filter on year and rating (FILTER clauses do not score)
Query filter = new BooleanQuery.Builder()
    .add(IntField.newRangeQuery("year", 2006, Integer.MAX_VALUE),
         Occur.FILTER)
    .add(FloatField.newRangeQuery("rating", 7.5f,
            Float.POSITIVE_INFINITY), Occur.FILTER)
    .build();

// 2. Vector query that applies the filter during the graph search
Query knn = new KnnFloatVectorQuery("embedding", query, 50, filter);

// 3. Hybrid query: vector score + BM25 on actors and genres
Query hybrid = new BooleanQuery.Builder()
    .add(knn, Occur.SHOULD)
    .add(new PhraseQuery("actors", "leonardo", "dicaprio"),
         Occur.SHOULD)
    .add(new TermQuery(new Term("genres", "thriller")),
         Occur.SHOULD)
    .add(filter, Occur.FILTER)
    .build();
TopDocs top = searcher.search(hybrid, 10);
```

▶ **Demo**: ch06-05-lucene-vector-search.ipynb

Key takeaways

- Vectors come from embeddings or perceptual features; compared by similarity (larger is better) or distance (smaller is better); the comparison belongs to the feature; normalization unifies cosine, dot product, and L2
- Voronoi, grid file, R-tree serve 2D/3D; the optimal NN algorithm needs only an optimistic bound per region, and follows several paths to prove its answer
- Curse of dimensionality: distances grow like √d with constant spread, contrast vanishes, dot products concentrate at 0; every leaf hits the NN-sphere
- "Nearest" loses meaning: clusters still stand out, but the exact NN is one good match among several; good matches suffice
- Three levers: fewer dimensions (PCA, Matryoshka), fewer bits (SQ, PQ, LSH), fewer vectors read (IVF, HNSW); combined two to three orders of magnitude, errors compound
- Recall against exact search understates quality: after reranking, cheap retrievers come close to exact; rerankers repair fine-filter errors, not coarse-filter errors
- In practice: FAISS, Lucene, OpenSearch, pgvector; vector search + keywords + filters; pre-, post-, or during-search filtering by selectivity and index; fuse with RRF

Further reading

- Guttman, A. (1984). **R-trees: a dynamic index structure for spatial searching**. *Proceedings of ACM SIGMOD*. ACM.
- Hjaltason, G. R., & Samet, H. (1999). **Distance browsing in spatial databases**. *ACM Transactions on Database Systems*, 24(2), 265-318. PDF.
- Ram, P., & Gray, A. G. (2012). **Maximum inner-product search using cone trees**. *Proceedings of ACM SIGKDD*, 931-939. arXiv preprint
- Weber, R., Schek, H.-J., & Blott, S. (1998). **A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces**. *Proceedings of VLDB*. PDF.
- Weber, R. (2001). **Similarity search in high-dimensional vector spaces**. Doctoral thesis, ETH Zurich, Diss. ETH No. 13974. *DISDBIS*, 74, Akademische Verlagsgesellschaft. ETH Research Collection
- Beyer, K., Goldstein, J., Ramakrishnan, R., & Shaft, U. (1999). **When is "nearest neighbor" meaningful?** *Database Theory: ICDT'99*, LNCS 1540, 217-235. Technical report PDF.
- Aggarwal, C. C., Hinneburg, A., & Keim, D. A. (2001). **On the surprising behavior of distance metrics in high-dimensional space**. *Proceedings of ICDT*, LNCS 1973. PDF.
- Weber, R., & Böhm, K. (2000). **Trading quality for time with nearest-neighbor search**. *Advances in Database Technology: EDBT 2000*, LNCS 1777, 21-35. Springer.
- Indyk, P., & Motwani, R. (1998). **Approximate nearest neighbors: towards removing the curse of dimensionality**. *Proceedings of ACM STOC*, 604-613. PDF.
- Charikar, M. S. (2002). **Similarity estimation techniques from rounding algorithms**. *Proceedings of ACM STOC*, 380-388. PDF.
- Malkov, Y. A., & Yashunin, D. A. (2020). **Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs**. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 824-836. arXiv preprint 2016.
- Jégou, H., Douze, M., & Schmid, C. (2011). **Product quantization for nearest neighbor search**. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1), 117-128. PDF.
- Voorhees, E., Alam, T., Bedrick, S., Demner-Fushman, D., Hersh, W. R., Lo, K., Roberts, K., Soboroff, I., & Wang, L. L. (2020). **TREC-COVID: constructing a pandemic information retrieval test collection**. *ACM SIGIR Forum*, 54(1), 1-12. arXiv preprint.